

Retrieval Augmented Generation with Knowledge Graphs

"Quick Summary"

- *GraphRAG is RAG where the retrieval path utilizes a knowledge graph.*
- *It improves the comprehensiveness and diversity of answers compared to RAG and it also enables questions/answers on more general and underlying themes of the data.*
- *Building the knowledge graph is the more complicated task than actually using GraphRAG.*

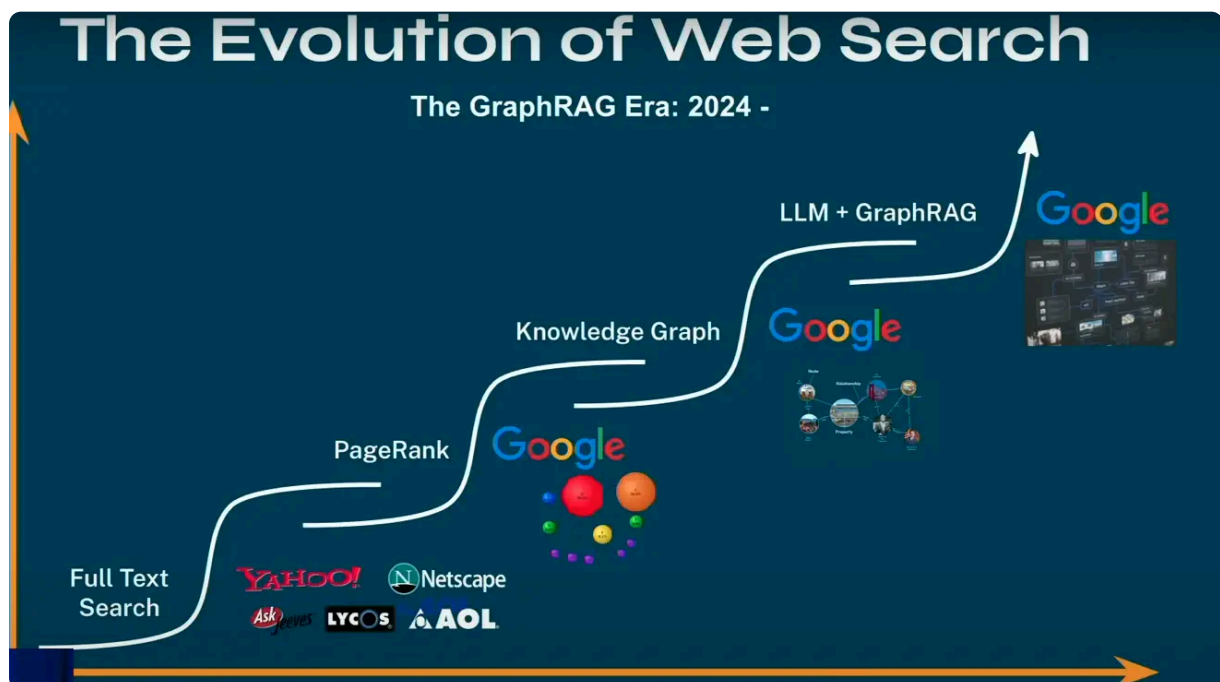


Figure 1: The evolution of Web Search. Google actually introduced the large scale use of a knowledge graph to enable the search engine to better understand what the user was searching for. Thus producing more relevant search results. In a way, GraphRAG is a combination of enabling a LLM to be fed relevant information from such a knowledge graph. Credits: This figure was taken from the presentation ¹

Graph RAG ²

Context

At the time of writing, [knowledge graphs](#) have been powering search across the web for around 12 years since [Google announced](#) in 2012 that the search experience would be enhanced by it. Knowledge graphs were meant to improve understanding of queries (especially some ambiguous queries) and to provide the user with a quick high-level summary of the things the user was possibly looking for. Also, it can be used to feed the user with possible new information that is linked in some way with the information on the subject he was searching for.

In the context of LLMs, naïve retrieval augmented generation (RAG) could be, roughly, the equivalent of Google before 2012 when no knowledge graph was involved. That would mean matching the user's query to the most relevant websites or the most similar documents in the knowledge base in the context of RAG. Of course, we oversimplify greatly for the purpose of the explanation, but the main idea of GraphRAG is to enhance the retrieval part of RAG by improving it the same way Google improved its search engine: using knowledge graphs to also provide some relevant and higher-level summaries of the information in the knowledge base corresponding to the user's query.

This way, GraphRAG can enable the LLM to capture global information about a corpus of text such as a general theme of a dataset. In some retrieval-focused tasks, the LLM can be empowered by looking through its knowledge base using similarity comparisons since it only needs to find information that's explicitly written somewhere in its knowledge base. However, some tasks require an understanding of the **relationships** of different parts of the knowledge base. For example, "What are the main themes in the dataset?" would be a query-focused summarization (QFS) task which would require the understanding of the relations between words of that dataset as the theme is often not explicitly written in a document.

The main idea is described as follows:

1. Derive an entity knowledge graph from the source documents.
2. Pre-generate community summaries for all groups of closely-related entities.
3. When the system receives a question: each community summary is used to generate a partial response.
4. Summarize all the partial responses from each community into a final response to the user.

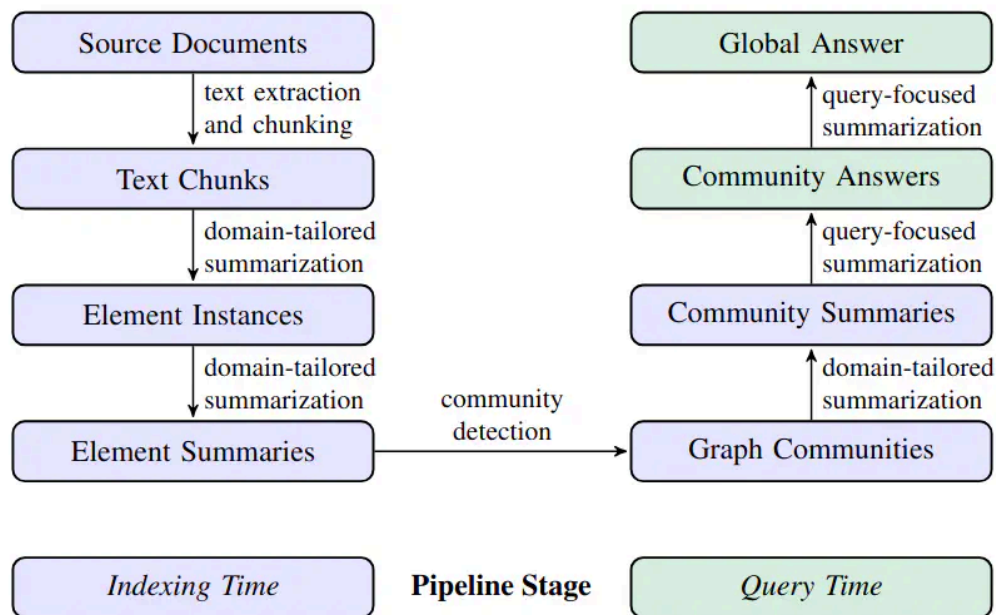


Figure 1: Graph RAG pipeline using an LLM-derived graph index of source document text. This index spans nodes (e.g., entities), edges (e.g., relationships), and covariates (e.g., claims) that have been detected, extracted, and summarized by LLM prompts tailored to the domain of the dataset. Community detection (e.g., Leiden, Traag et al., 2019) is used to partition the graph index into groups of elements (nodes, edges, covariates) that the LLM can summarize in parallel at both indexing time and query time. The “global answer” to a given query is produced using a final round of query-focused summarization over all community summaries reporting relevance to that query.

Method

As most RAG methods³, this method starts with a stage of indexing which basically consists of organizing the knowledge base (in advance, before deploying the system) in a knowledge graph. The second stage of that method is what’s executed once the system receives a query from the user. That process is separated in two different workflows that are detailed below.

Indexing (precalculated)

1. Source Documents -> Text Chunks

- As in naive RAG, we separate each document in different chunks. The smaller the chunks, the more detailed information about the documents we save, but with a higher number of nodes in our graph (or database in the case of naive RAG).

2. Text Chunks -> Element Instances -> Element summaries

- Extract **entities** (persons, places, objects, etc.), **relationships** and **key claims** (**facts, affirmations**) of each chunk.

- Ask the LLM using multiple *gleaning* (extraction) rounds to identify instances until it has affirmed that not many other instances could be identified.
- Output: subgraph-per-chunk containing a list of **entities** (*name, type, description*) and a list of **relationships** (*source, target, description*).
- Remove duplicates in each list by merging their attributes/descriptions.
- **Summarize** entity, relationships and claims.
 - **Ask the LLM** for a short summary that captures all the distinct information from the description of each entity/relationship to have a single concise description of them.

3. Augment the graph

1. Community detection

- Perform hierarchical clustering on the initial knowledge graph using **Leiden** technique to identify communities. This applies a recursive community-clustering to the graph until a community-size threshold has been reached.
- Multiple levels and hierarchy of communities will be identified providing a structured way to navigate through the graph.

2. Graph Embedding

- Transform the graph into a vector representation. Using the Node2Vec algorithm, each node is represented as a vector in a lower-dimensional space where adjacent nodes in the graph are represented by vectors close to each other.
- Useful vector-space in which to search for during the query phase.

4. Community summarization

1. Generate Community Reports

- Ask the LLM to generate a summary of each community (at all levels of the hierarchy identified in step 3.1) to provide the knowledge base with a scoped understanding of the graph at different granularity.

2. Summarize Community Reports

- Ask the LLM to summarize each report of the communities for shorthand use.

3. Community Embedding

- Generate a **vector representation** of the **communities** by generating text embeddings of: the community report + report summary + title.
- Useful vector-space in which to search for during the query phase.

Querying (at runtime)

Two types of workflows are proposed to answer different queries. Those two types of workflows relate directly to retrieval-focused and query-focused summarization tasks that was mentioned above. To answer retrieval-focused tasks, which are questions that require an understanding of specific entities that are *mentioned* in the documents, the *local search* workflow would be the pipeline that would enhance an answer to such a question which is what most RAG pipeline aim to do. On the other hand, to answer questions that require a more *general understanding* of the knowledge base (e.g. the theme), the *global search* is meant to help with that, which was the core idea behind the paper.

Local search

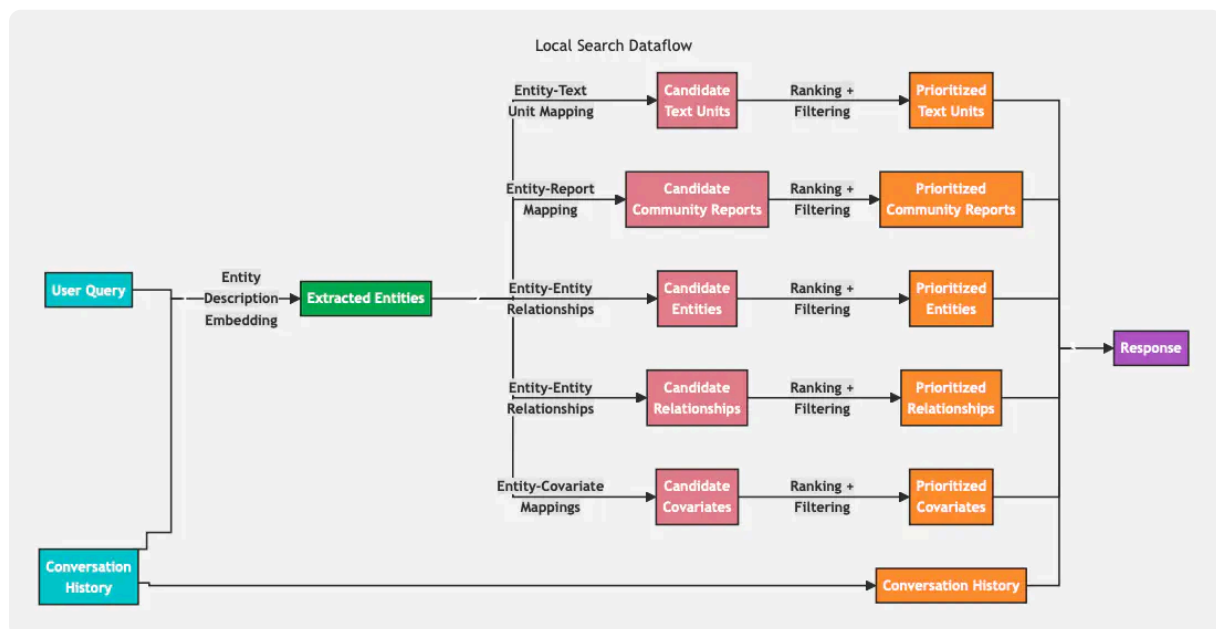


Figure 2: Local search dataflow. Source: Microsoft Research [4](#)

This local search method identifies a set of entities from the knowledge graph that are **semantically-related** to the user's query (also considering the conversation history). Those entities serve as access point into the knowledge graph to extract other information such as neighbouring entities, relationships, claims, community reports (of any level) and relevant raw text chunks that are associated with those entities. All of these candidate source of information are ranked and filtered to fit within a window of pre-defined maximum number of tokens, including only the most relevant information to be used to generate an answer to the user.

Global search

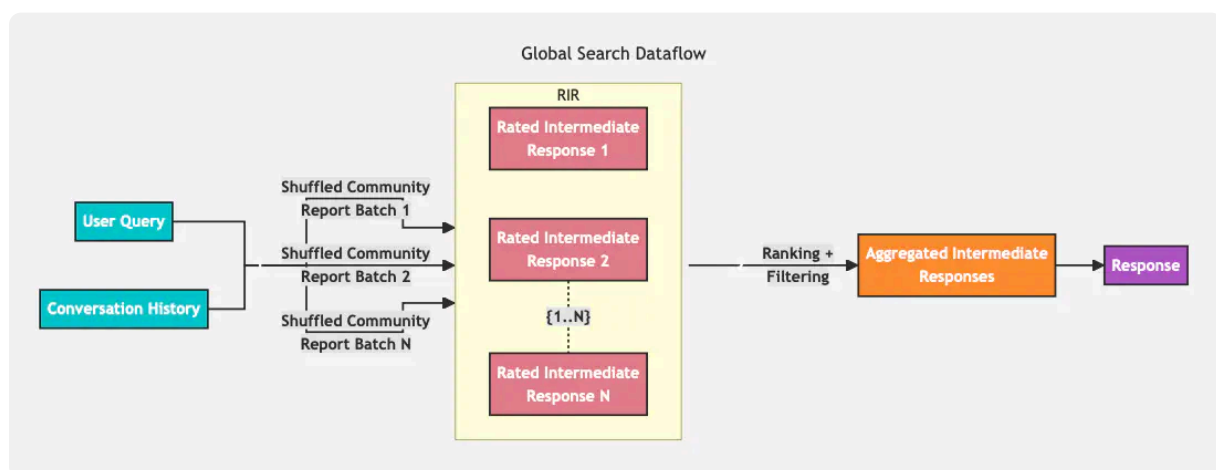


Figure 3: Global search dataflow. Source: Microsoft Research [4](#)

This global search approach is the most well-suited workflow to handle tasks that require a broader understanding of the knowledge graph. The community summaries generated during the indexing pipeline are here used to general an answer in a multi-stage process. This workflow utilizes a map-reduce approach:

1. Prepare community summaries.

- Randomly shuffle and divide the community summaries into several pre-defined chunk sizes. We want the information of the community summaries to be distributed across the different chunks (called “Shuffle Community Report Batch N” in the image).

2. Map community answers.

- For each chunk, generate an intermediate answer in parallel based on its respective shuffled community report chunk.

- Also, ask the LLM to evaluate how helpful the generated answer is in answering the target question on a value between 0 and 100. Answers with score=0 are filtered out.

3. **Reduce** to global answer.

- Sort the intermediate answers in descending order from most to least useful based on the score given by the LLM in the previous step.
- Add all the most useful intermediate answers that fit in the context window that will then be used to generate the global answer to the user.

Benchmark

Datasets:

1. Compiled podcast transcripts from *Behind the Tech*, by Kevin Scott. ~1M tokens.
2. New articles published from September 2013 to December 2023 in a range of categories (entertainment, business, sports, tech, health and science). ~1.7M tokens.

Global questions

Since the paper primarily focuses on the *global search*, the objective was to evaluate the effectiveness of RAG systems for more global sensemaking tasks. They asked the LLM to generate N questions that require the understanding of the entire corpus.

Conditions (four levels of graph communities to compare)

C0: Using root-level community summaries (fewest in numbers).

C1: Using high-level community summaries. Sub-communities of C0.

C2: Using intermediate-level community summaries. Sub-communities of C1.

C3: Using low-level community summaries (greatest in numbers). Sub-communities of C2.

TS: Source texts (instead of community summaries) are shuffled and chunked for the map-reduce summarization stages.

SS: Naïve RAG.

Results

- Global approaches consistently out-perform the naïve RAG approach in both **comprehensiveness** and **diversity** metrics for a win rate around 72% on average.
- Naïve RAG produces the most direct responses (50-60% win rate for directness) across all comparisons.
- Community summaries generally provided a small but consistent improvement in comprehensiveness and diversity (except for C0).
- Empowerment showing mixed results. Tuning element extraction prompts may help.

Podcast transcripts																											
Comprehensiveness							Diversity							Empowerment							Directness						
	SS	TS	C0	C1	C2	C3		SS	TS	C0	C1	C2	C3		SS	TS	C0	C1	C2	C3		SS	TS	C0	C1	C2	C3
SS	50	17	28	25	22	21	SS	50	18	23	25	19	19	SS	50	42	57	52	49	51	SS	50	56	65	60	60	60
TS	83	50	50	48	43	44	TS	82	50	50	50	43	46	TS	58	50	59	55	52	51	TS	44	50	55	52	51	52
C0	72	50	50	53	50	49	C0	77	50	50	50	46	44	C0	43	41	50	49	47	48	C0	35	45	50	47	48	48
C1	75	52	47	50	52	50	C1	75	50	50	50	44	45	C1	48	45	51	50	49	50	C1	40	48	53	50	50	50
C2	78	57	50	48	50	52	C2	81	57	54	56	50	48	C2	51	48	53	51	50	51	C2	40	49	52	50	50	50
C3	79	56	51	50	48	50	C3	81	54	56	55	52	50	C3	49	49	52	50	49	50	C3	40	48	52	50	50	50

News articles																											
Comprehensiveness							Diversity							Empowerment							Directness						
	SS	TS	C0	C1	C2	C3		SS	TS	C0	C1	C2	C3		SS	TS	C0	C1	C2	C3		SS	TS	C0	C1	C2	C3
SS	50	20	28	25	21	21	SS	50	33	38	35	29	31	SS	50	47	57	49	50	50	SS	50	54	59	55	55	54
TS	80	50	44	41	38	36	TS	67	50	53	45	44	40	TS	53	50	58	50	50	48	TS	46	50	55	53	52	52
C0	72	56	50	52	54	52	C0	62	47	50	40	41	41	C0	43	42	50	42	45	44	C0	41	45	50	48	48	47
C1	75	59	48	50	58	55	C1	65	55	60	50	50	50	C1	51	50	58	50	52	51	C1	45	47	52	50	49	49
C2	79	62	46	42	50	59	C2	71	56	59	50	50	51	C2	50	50	55	48	50	50	C2	45	48	52	51	50	49
C3	79	64	48	45	41	50	C3	69	60	59	50	49	50	C3	50	52	56	49	50	50	C3	46	48	53	51	51	50

Figure 4: Head-to-head win rate percentages of (row condition) over (column condition) across two datasets, four metrics, and 125 questions per comparison (each repeated five times and averaged). The overall winner per dataset and metric is shown in bold. Self-win rates were not computed but are shown as the expected 50% for reference. All Graph RAG conditions outperformed naïve RAG on comprehensiveness and diversity. Conditions C1-C3 also showed slight improvements in answer comprehensiveness and diversity over TS (global text summarization without a graph index).

The following Table 3 illustrates the scalability advantages of such a method. Utilizing higher-level community summaries allows to use significantly less tokens than utilizing raw document summarization which utilized the most resources to provide the answer. For low-level community summaries (C3), it required 26-33% fewer tokens while for root-level community summaries (C0) it required **97% fewer tokens!**

	Podcast Transcripts					News Articles				
	C0	C1	C2	C3	TS	C0	C1	C2	C3	TS
Units	34	367	969	1310	1669	55	555	1797	2142	3197
Tokens	26657	225756	565720	746100	1014611	39770	352641	980898	1140266	1707694
% Max	2.6	22.2	55.8	73.5	100	2.3	20.7	57.4	66.8	100

Table 3: Number of context units (community summaries for **C0-C3** and text chunks for **TS**), corresponding token counts, and percentage of the maximum token count. Map-reduce summarization of source texts is the most resource-intensive approach requiring the highest number of context tokens. Root-level community summaries (**C0**) require dramatically fewer tokens per query (9x-43x).

References

1. “GraphRAG: The Marriage of Knowledge Graphs and RAG: Emil Eifrem” viewed on Sept. 16th 2024. <https://www.youtube.com/watch?v=knDDGYHnnSI>. ↵
2. Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., ... & Larson, J. (2024). From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*. ↵
3. <https://docs.nvidia.com/ai-enterprise/workflows-generative-ai/0.1.0/technical-brief.html> ↵
4. <https://microsoft.github.io/graphrag/> ↵ ↵